



PROYECTO INTERMODULAR

2ºDAW

TERRETA HOUSE



IES ABASTOS



**GENERALITAT
VALENCIANA**

Conselleria d'Educació,
Cultura i Esport

CARLOS GUIJARRO MINGUEZ
CFGS Desarrollo de Aplicaciones Web (DAW)
Curso académico (2025–2026)

Contenido

Resumen del proyecto	2
Justificación y objetivos	3
Desarrollo del Proyecto.....	4
Análisis del mercado	4
Metodología	5
Diseño de la aplicación	6
Tecnologías utilizadas.....	8
Análisis de componentes.....	8
Herramientas de soporte y Entorno	9
Diagramas UML — TerretaHouse	10
Diagrama de Casos de Uso	10
ESQUEMA Entidad-Relación	11
Guía de Estilos — TerretaHouse	12
Concepto y Filosofía de Diseño.....	12
Paleta Cromática.....	12
Tipografía.....	13
Estrategia de Diseño Responsive.....	14
UX	14
Protocolos de Seguridad y Validación.....	15
Autenticación basada en Tokens.....	15
Rutas API	16
Análisis de Proceso, Retos y Líneas de Evolución.....	17
Retos superados	17
Lo más positivo del proyecto	17
Líneas de mejora	17
Reflexión final.....	18
Bibliografía.....	19
Documentación y Frameworks	19

Resumen del proyecto

TerretaHouse es una plataforma web integral diseñada para la búsqueda, visualización y reserva de **actividades y experiencias turísticas** en la Comunitat Valenciana. El objetivo principal de la aplicación es facilitar al usuario la exploración de propuestas de ocio ubicadas en entornos naturales y culturales, proporcionando información detallada sobre cada actividad, incluyendo descripción, fecha, disponibilidad y plazas.

Además de la gestión de reservas, la plataforma integra contenido complementario relacionado con el turismo regional, permitiendo descubrir destinos de interés como pueblos con encanto, zonas de montaña y espacios protegidos. El sistema ha sido desarrollado bajo un enfoque centrado en la usabilidad y la experiencia de usuario (UX), garantizando una interfaz adaptada a cualquier dispositivo gracias a un diseño *responsive* y una arquitectura basada en componentes reutilizables. La plataforma implementa un control de acceso segmentado, permitiendo que visitantes, usuarios registrados y administradores gestionen de forma eficiente tanto la oferta de actividades como el flujo completo de reservas.

Versión en inglés

TerretaHouse is a comprehensive web platform designed for searching, viewing and booking tourist activities and experiences in the Valencian Community. The main aim of the application is to make it easier for users to explore leisure options set in natural and cultural environments, providing detailed information on each activity, including a description, dates, availability and number of places.

In addition to managing bookings, the platform features supplementary content related to regional tourism, allowing users to discover destinations of interest such as charming villages, mountain areas and protected sites. The system has been developed with a focus on usability and user experience (UX), ensuring an interface adapted to any device thanks to a responsive design and an architecture based on reusable components. The platform implements segmented access control, allowing visitors, registered users and administrators to efficiently manage both the range of activities on offer and the entire booking process.

Justificación y objetivos

El sector de las actividades de ocio y turismo activo en la Comunitat Valenciana tiene un potencial enorme, pero a menudo carece de un punto de encuentro digital que simplifique la reserva de experiencias locales. Al analizar el mercado actual, detecté que el usuario que busca disfrutar de una ruta, una visita guiada o una actividad de aventura en nuestra tierra suele encontrarse con un proceso fragmentado: información dispersa, dificultades para confirmar plazas en tiempo real o una gestión de reservas poco intuitiva.

TerretaHouse nace para cubrir este vacío. Mi intención no es competir con los grandes portales de ocio global, sino ofrecer una alternativa especializada que ponga en valor las experiencias locales valencianas. La motivación detrás de este proyecto es doble: por un lado, crear una herramienta que conecte de forma eficaz al usuario con actividades que enriquecen su estancia; y por otro, consolidar mis competencias técnicas integrando, de principio a fin, una arquitectura completa de desarrollo web, desde el frontend hasta la persistencia de datos en el backend.

Objetivos del proyecto

- Implementar un sistema de autenticación seguro: Garantizar la privacidad y gestión de perfiles mediante un modelo de acceso controlado.
- Desarrollar un catálogo de experiencias dinámico: Crear un sistema eficiente de búsqueda y visualización de actividades turísticas disponibles.
- Optimizar el ciclo de vida de la reserva: Diseñar un flujo de trabajo intuitivo que permita a los usuarios realizar, consultar y gestionar sus plazas de forma fluida.
- Poner en valor el entorno local: Integrar información sobre destinos y actividades en la Comunitat Valenciana, convirtiendo la plataforma en una guía de experiencias.
- Garantizar una experiencia multiplataforma: Asegurar una interfaz 100% responsive, priorizando la usabilidad tanto en dispositivos móviles como en equipos de escritorio.
- Aplicar una arquitectura profesional: Desplegar una solución basada en el patrón cliente-servidor, asegurando la separación de responsabilidades entre el frontend (Angular) y el backend (PHP).
- Consolidar el aprendizaje técnico: Aplicar de forma práctica los contenidos del ciclo formativo, resolviendo retos reales de desarrollo, seguridad y gestión de datos.



Desarrollo del Proyecto

Análisis del mercado

Hoy en día, cuando alguien quiere reservar una actividad de ocio o una visita turística, lo primero que hace es entrar en plataformas como TripAdvisor, Booking o Airbnb. Son webs muy potentes, pero tienen un problema: están pensadas para una escala global. Al final, lo que ofrecen es lo mismo para todo el mundo y, a menudo, las actividades locales o los pequeños organizadores de la Comunitat Valenciana se pierden entre miles de anuncios internacionales.

TerretaHouse nace precisamente para solucionar eso: es una plataforma enfocada 100% en el ocio dentro de nuestra región. En lugar de intentar abarcar todo el planeta, prefiero centrarme en lo que tenemos cerca. El objetivo es dar visibilidad a quienes organizan rutas, visitas o actividades de aventura aquí, facilitando que el usuario encuentre planes reales y cercanos, sin tanta información irrelevante que suele aparecer en otras webs más grandes.

Además de gestionar las reservas, el proyecto busca ser algo más útil. He querido integrar información turística que ayude al usuario a elegir mejor qué hacer, ofreciendo guías y recomendaciones que den sentido a la reserva. No es solo "reservar y ya", es poder planificar un plan de ocio completo.

Sobre cómo podría ganar dinero en un futuro (modelo de negocio), aunque ahora mismo mi meta es que la plataforma funcione perfectamente y sea funcional, creo que tiene mucho potencial. Podría crecer mediante comisiones por cada reserva gestionada, haciendo acuerdos con los propios organizadores de las actividades o incluso creando un espacio para promocionar eventos especiales en la Comunitat. De momento, mi prioridad ha sido demostrar que el sistema funciona y que la tecnología detrás es sólida.



Metodología

Para organizar el desarrollo de TerretaHouse, no he seguido un proceso rígido, sino que he preferido ir paso a paso. Me ha funcionado muy bien trabajar por fases: primero lo básico y, una vez que eso funcionaba, ir añadiendo capas de funcionalidad.

La primera parte fue la más mental: sentarme a pensar qué necesitaba realmente la web. Definí los objetivos clave y qué funcionalidades eran imprescindibles para que el usuario pudiera reservar una actividad sin complicaciones. Con eso claro, me metí en Figma para diseñar la interfaz. Hice unos *wireframes* básicos; no quería perder demasiado tiempo en colores o detalles estéticos al principio, mi prioridad era que la estructura de la web fuera fácil de usar y que el usuario supiera en todo momento dónde estaba.



Cuando tuve el diseño claro, empecé a programar. Decidí separar totalmente el desarrollo del *frontend* y el *backend*. Para mí ha sido la mejor decisión porque me ha permitido centrarme en una cosa cada vez: primero dejar la API lista en PHP, probar que devolviera bien los datos y, luego, conectar el *frontend* con Angular. Si me salía un error, sabía exactamente dónde buscar, lo que me ha ahorrado bastantes dolores de cabeza.

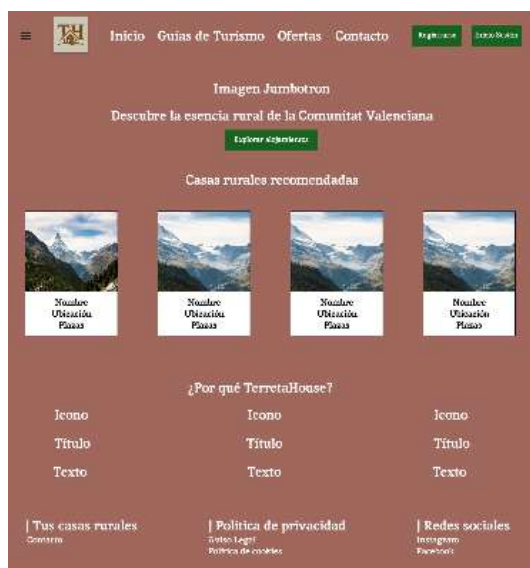
Además, he usado Git para todo el control de versiones. Es algo que, para todo programador, me parece obligatorio; me ha dado mucha tranquilidad saber que, si rompía algo durante un cambio, siempre podía volver atrás a un punto donde todo funcionaba bien, aunque a veces entre tanto fichero me volvía un poco loco. En resumen, ha sido un trabajo de ir construyendo bloques: primero la BBDD, luego la comunicación con el servidor y finalmente el diseño final. Así he conseguido que el proyecto fuera creciendo de forma ordenada y sin que el código se convirtiera en un caos.

Diseño de la aplicación

Antes de escribir ni una sola línea de código, tuve claro que necesitaba un mapa de ruta. Dediqué tiempo a diseñar la interfaz en Figma porque, al final, es mucho más rápido cambiar un botón de sitio en un diseño que reescribir componentes en Angular.

Empecé creando unos *wireframes* muy básicos. Mi objetivo no era hacer algo bonito todavía, sino que fuera funcional. Me centré en dónde poner la barra de navegación(navbar), cómo organizar el catálogo de actividades y dónde colocar los botones de reserva para que el usuario no tuviera que dar mil vueltas para encontrar lo que busca. Entonces, sabía que debía priorizar la usabilidad y que la estructura fuera intuitiva; si la base es lógica, el resto suele ir un poco rodado.

Al final, de la idea que tenía al principio a lo que ha acabado siendo el proyecto final, han cambiado muchas cosas. A medida que avanzaba en el desarrollo, me di cuenta de que algunos planteamientos iniciales no terminaban de encajar bien en la práctica. En lugar de forzar el diseño original, preferí hacer ajustes sobre la marcha, simplificando procesos y adaptando la interfaz a lo que el usuario realmente necesita al navegar. Esa capacidad de pivotar y ajustar el diseño original a lo que el proyecto pedía ha sido uno de los aprendizajes más valiosos de este trabajo



La web gira en torno a varias páginas clave: el inicio, dentro el catálogo de las actividades, una sección de guías turísticas, la zona de contacto y, por supuesto, todo el flujo de inicio de sesión y registro. He intentado que todas sigan un mismo estilo para que, cuando el usuario navegue, no sienta que está cambiando de web, sino que todo se mantenga coherente.



Por último, me aseguré de que el diseño fuera totalmente *responsive*. Hoy en día, la mayoría de la gente consulta planes de ocio desde el móvil mientras está en la calle, así que era obligatorio que la web se viera igual de bien en un ordenador, en una Tablet o en un teléfono pequeño, sin que se rompiera el diseño o fuera complicado hacer clic en los botones.



Tecnologías utilizadas

Para el desarrollo de TerretaHouse, he seleccionado un “camino” tecnológico que me permitiera trabajar de forma organizada y, sobre todo, que garantizara que la aplicación fuera robusta y escalable.

Capa	Tecnología	Justificación
Frontend	Angular + Bootstrap	Angular aporta robustez mediante TypeScript, facilitando la escalabilidad del código. Bootstrap agiliza el diseño responsive.
Backend	PHP Nativo	Gestión de lógica de servidor y seguridad sin abstracciones, permitiendo un control total sobre las peticiones.
Persistencia	MySQL	Sistema de gestión de bases de datos relacionales, ideal para la integridad referencial de usuarios y reservas.

Análisis de componentes

- **Frontend (Angular & Bootstrap):** He trabajado con Angular aprovechando la tipado fuerte de **TypeScript**. Esto ha reducido drásticamente los errores en tiempo de ejecución al estructurar los datos del usuario. El uso de Bootstrap ha sido vital para el diseño responsive, permitiendo implementar una rejilla adaptable que responde a diferentes vistas sin necesidad de reinventar la hoja de estilos desde cero.
- **Backend (API en PHP Nativo):** Esta ha sido la parte más exigente y, a la vez, la que más valor ha aportado a mi perfil profesional. Al desarrollar la API en PHP nativo, he tenido que implementar manualmente el tratamiento de las cabeceras HTTP, la gestión de sesiones y la seguridad de las rutas. Esta experiencia me ha brindado una comprensión profunda de cómo funciona el protocolo HTTP y la comunicación cliente-servidor más allá de lo que ofrece cualquier framework.

Herramientas de soporte y Entorno

El flujo de trabajo se ha apoyado en herramientas estándar de la industria que garantizan la calidad del desarrollo:

Entorno de desarrollo: Visual Studio Code ha sido mi editor principal, complementado con extensiones para el manejo de SQL y depuración de código PHP.

Control de versiones (Git/GitHub): El uso de Git no ha sido solo para guardar código, sino para gestionar un flujo de trabajo organizado. Me ha permitido ramificar el desarrollo de nuevas funcionalidades y, lo más importante, disponer de un histórico de versiones que ha actuado como red de seguridad ante cualquier fallo inesperado.

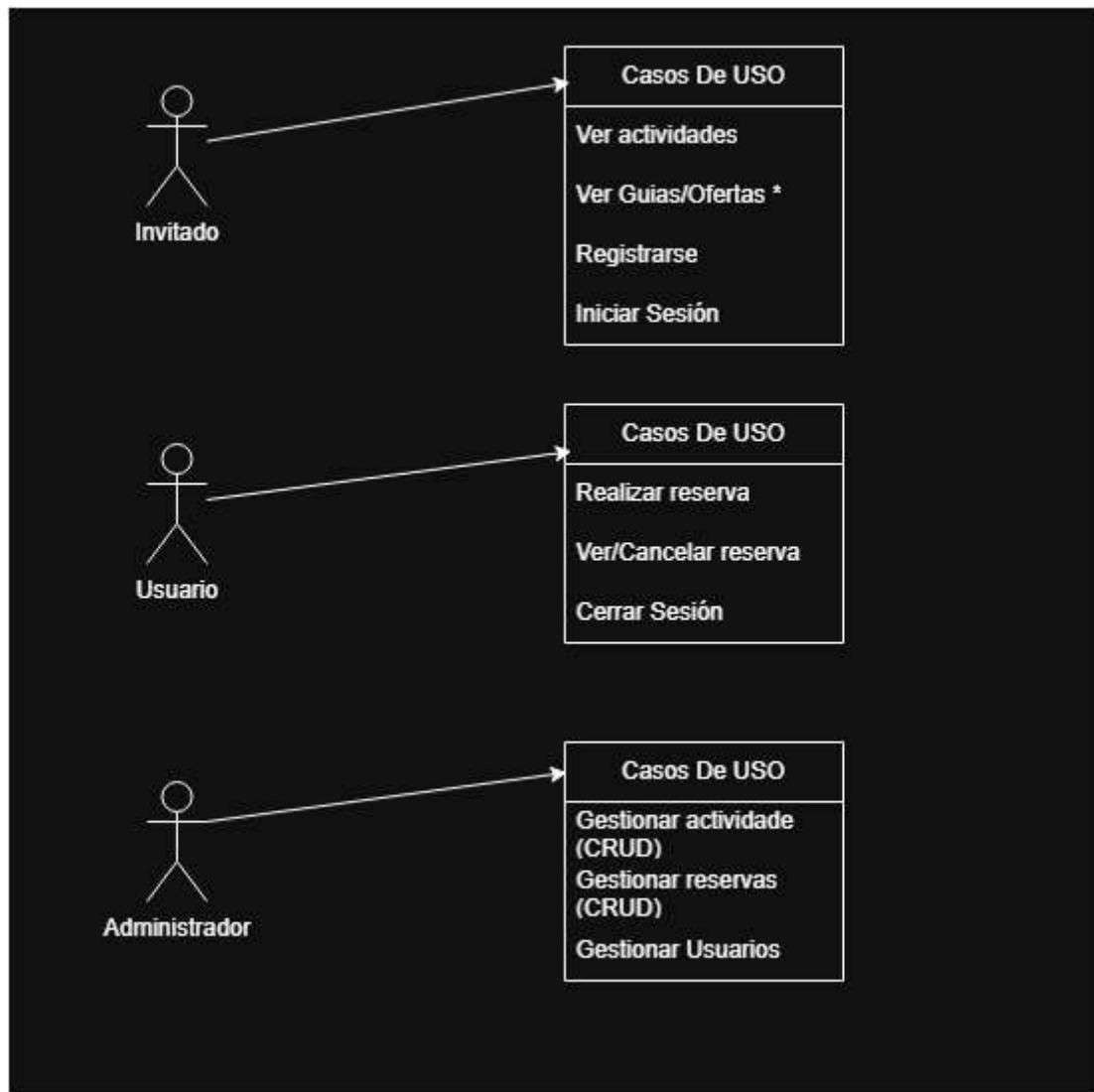
Pruebas de API (Postman): Antes de integrar cualquier llamada en Angular, he validado cada *endpoint* en Postman. Esto me ha permitido asegurar que los datos viajan correctamente entre la base de datos y la interfaz, depurando problemas de formato JSON antes de que llegaran al frontend.

Contenedores Despliegue (Docker): Para asegurar la portabilidad del proyecto, he trabajado con Docker. Al crear contenedores separados para el servidor y la base de datos, he eliminado el problema de "en mi máquina funciona", permitiendo que el entorno de desarrollo sea idéntico en cualquier sistema operativo.



Diagramas UML — TerretaHouse

Diagrama de Casos de Uso



Este diagrama muestra cómo interactúan los usuarios con TerretaHouse dependiendo de su rol. He organizado los permisos de forma sencilla: el **Invitado** puede moverse libremente por la web para descubrir actividades, pero necesita registrarse para dar el siguiente paso. Una vez que el **Usuario** inicia sesión, ya tiene acceso a poder reservar y gestionar sus propias actividades. Por último, he dejado reservadas todas las tareas de mantenimiento —como añadir nuevas actividades, borrar registros o gestionar usuarios— al perfil de **Administrador**. Con esta estructura, no solo consigo que la navegación sea intuitiva, sino que también aseguro que la información esté protegida, evitando que alguien sin permisos pueda acceder a funciones o datos que no le corresponden.

ESQUEMA Entidad-Relación

Para estructurar la base de datos, he definido tres tablas principales que permiten gestionar de forma sencilla tanto a los usuarios como las actividades y el historial de reservas.

Esta es mi estructura:

Tabla USER (Usuarios): Gestiona el acceso de los usuarios al sistema.

- id: INT (PK(clave primaria), AUTO_INCREMENT).
- email: VARCHAR (garantiza la exclusividad de cada usuario).
- password: VARCHAR (almacenada de forma encriptada).
- roles: JSON (almacena el rol para distinguir entre usuario y administrador).
- api_token: VARCHAR(utilizado para la gestión de sesiones)

Tabla ACTIVIDAD (Catálogo de eventos): Define el catálogo de planes disponibles.

- id: INT (PK).
- título: VARCHAR (Nombre de la actividad).
- fecha: DATE (Fecha programada).
- estado: VARCHAR(indica si la actividad está activa o deshabilitada).

Tabla RESERVA: Es la tabla que articula la relación entre usuarios y actividades.

- id: INT (PK).
- título: VARCHAR.
- fecha: DATE.
- estado: VARCHAR.
- user_id: (FK referenciada a user).
- Actividad_id: (FK referenciada a actividad)

Después en cuando a la lógica de negocio me he basado en dos relaciones de uno a muchos (1:N).

Usuario - Reserva: Un usuario puede tener múltiples reservas. Cada registro en reserva está ligado a un user_id. He configurado una restricción ON DELETE CASCADE, lo que significa que, si un usuario es eliminado de la plataforma, todas sus reservas asociadas se eliminan automáticamente, evitando registros huérfanos.

Actividad - Reserva: Del mismo modo, cada reserva apunta a una única actividad (actividad_id). También en este caso he vuelto aplicar ON DELETE CASCADE, garantizando que si una actividad se elimina del sistema, no queden reservas activas vinculadas a un recurso que ya no existe.

Guía de Estilos — TerretaHouse

Concepto y Filosofía de Diseño

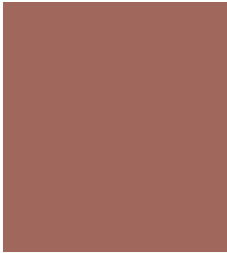
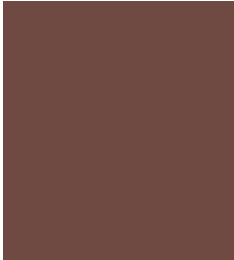
Cuando empecé a pensar en el diseño de TerretaHouse, tenía claro que no quería hacer otra web de reservas más, de esas que parecen frías y cargadas de información. Lo que yo buscaba es que, nada más entrar en la página, el usuario sintiera esa misma calma y desconexión que buscas cuando planeas una escapada rural.

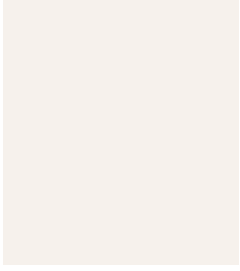
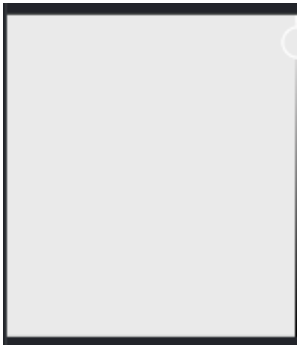
Para lograr esa sensación de 'acogida', me centré en tres cosas muy básicas pero que marcan la diferencia:

- **Que se lea fácil:** No sirve de nada tener una web bonita si el usuario tiene que forzar la vista. He elegido fuentes muy limpias que facilitan la lectura rápida, para que encontrar la información sea sencillo y no canse.
- **Que el usuario sepa dónde hacer clic:** He usado los colores para crear una guía visual. Básicamente, he diseñado la web para que, casi sin darse cuenta, el ojo se vaya directo a los botones de reserva. No quería que el usuario tuviera que perder tiempo adivinando cómo funciona la plataforma.
- **Que funcione en el móvil sí o sí:** He diseñado bajo la mentalidad de 'móvil primero'. Al final, cuando alguien está de viaje o buscando un plan de fin de semana, casi nunca tiene un ordenador delante; lo hace desde el móvil mientras está en la calle. Por eso, me he asegurado de que, tanto si abres la web en un teléfono pequeño como si lo haces en un monitor grande, la experiencia sea igual de fluida y fácil.

Paleta Cromática

La elección de colores busca la armonía con el entorno rural. He definido una paleta de tonos cálidos y naturales que facilitan la navegación sin fatigar la vista.

Color	Código HEX	Aplicación en el Proyecto
Marrón Tierra 	#A0685D	Color principal. Usado en elementos de llamada a la acción (botones primarios), headers y footers.
Marrón Oscuro 	#6E4A42	Aplicado en estados <i>hover</i> de botones y títulos principales para generar contraste y profundidad.

Beige Claro 	#F6F1EC	Color de fondo principal. Aporta calidez y reduce la fatiga visual frente al blanco puro.
Blanco 	#FFFFFF	Fondo para tarjetas (cards) y contenedores de formularios para destacar el contenido.
Gris Claro 	#EAEAEA	Bordes, separadores y sombras suaves, manteniendo la estructura limpia.

Tipografía

La selección tipográfica responde a la necesidad de modernidad y limpieza. He utilizado dos fuentes de Google Fonts, ambas integradas mediante importación CSS:

Poppins (Principal): Es una fuente geométrica que ofrece una legibilidad excelente tanto en titulares como en párrafos. Su uso principal es en los cuerpos de texto y descripciones de las actividades, donde se requiere claridad.

Montserrat (Alternativa): Utilizada principalmente para encabezados y botones de navegación. Aporta un toque más rotundo y profesional, reforzando la jerarquía visual de la página.

Justificación técnica: Ambas fuentes han sido seleccionadas por su excelente renderizado en pantallas de alta resolución y su gran rendimiento de carga en la web.

Estrategia de Diseño Responsive

El diseño *responsive* es una pieza clave en TerretaHouse, dado que el usuario tipo suele consultar disponibilidad mientras se desplaza. Para ello, he implementado una rejilla flexible utilizando **Bootstrap** junto con media “queries” personalizadas.

Los **breakpoints** definidos son:

Dispositivos móviles (hasta 768px): Diseño en columna única para maximizar el área de interacción táctil.

Tablets (769px a 1024px): Reajuste de la rejilla a dos columnas, permitiendo visualizar más contenido sin saturar la pantalla.

Desktop (más de 1024px): Aprovechamiento total del ancho de pantalla con una estructura optimizada en rejilla de 3 o 4 columnas, facilitando la comparación de alojamientos.



UX

Para mejorar la experiencia, he implementado los siguientes patrones de diseño:

Consistencia: El uso de estilos repetitivos en botones y tarjetas de actividades asegura que el usuario no necesite "reaprender" la interfaz al cambiar de página.

Espaciado (White Space): He dejado intencionadamente "aire" entre los elementos para que el usuario pueda escanear la información rápidamente.

Feedback visual: Mediante el uso de estados :hover y transiciones suaves en CSS, el usuario recibe confirmación inmediata de que está interactuando con elementos activos.

Protocolos de Seguridad y Validación

La seguridad en TerretaHouse no ha sido un componente añadido al final, sino un pilar sobre el que he construido toda la API. Entender que el cliente (Angular) y el servidor (PHP) son dos mundos independientes me ha obligado a ser extremadamente cuidadoso con cada dato que entra y sale de la base de datos.

Protección contra Inyecciones SQL

El riesgo más importante al trabajar con MySQL es la inyección SQL. Para neutralizarlo, he abandonado el uso de consultas directas y he implementado **sentencias preparadas**. Al separar la consulta SQL de los datos proporcionados por el usuario, el motor de la base de datos trata cualquier entrada como una cadena de texto, impidiendo que comandos maliciosos puedan ser ejecutados en el servidor.

Gestión de Identidad y Encriptación

El manejo de las contraseñas sigue los estándares actuales de seguridad. Nunca almaceno contraseñas en texto plano. He utilizado la función **password_hash()** de PHP, que aplica un algoritmo de hashing. Esto garantiza que, incluso si el archivo de la base de datos fuera comprometido, las contraseñas serían prácticamente imposibles de revertir.

Autenticación basada en Tokens

Para mantener la sesión del usuario registrada sin exponer datos sensibles, he optado por un sistema de `api_token`. Tras un inicio de sesión correcto, el servidor genera un token único que el cliente guarda en el almacenamiento local. Cada vez que el usuario intenta realizar una acción sensible (como reservar una actividad), debe enviar este token en la cabecera de la petición. Si el token no coincide con el registrado en la base de datos para ese usuario, el servidor deniega el acceso automáticamente.

Rutas API

Método	Ruta	Acción	Descripción
GET	/api/actividades.php	Listado	Recupera el catálogo de actividades disponibles.
POST	/api/login.php	Autenticación	Valida credenciales y devuelve el objeto usuario con api_token.
POST	/api/register.php	Registro	Da de alta un nuevo usuario en la base de datos.
POST	/api/logout.php	Cierre sesión	Invalida la sesión y el token del usuario actual.
POST	/api/reservar.php	Creación	Registra una reserva vinculando user_id y actividad_id.
GET	/api/get_reservas.php	Consulta	Obtiene el historial de reservas según el usuario autenticado.
POST	/api/delete_reserva.php	Cancelación	Elimina o marca como cancelada una reserva específica.
POST	/api/admin/add_act.php	Gestión	Crea nuevas actividades.
POST	/api/admin/del_act.php	Gestión	Elimina actividades del catálogo.

Análisis de Proceso, Retos y Líneas de Evolución

El desarrollo de mi app TerretaHouse no ha sido una línea recta, sino un proceso de aprendizaje constante. Mirando atrás, me doy cuenta de que este proyecto ha sido mucho más que escribir código; ha sido entender cómo las piezas de un sistema se conectan entre sí y, sobre todo, cómo reaccionar cuando algo no sale como estaba previsto en los bocetos iniciales, cosa que me ha ocurrido constantemente.

Retos superados

La parte que, sin duda, más me ha hecho sudar ha sido el desarrollo del backend en PHP nativo. Al no contar con las facilidades que ofrecen los frameworks (como Laravel o Symfony), me encontré cara a cara con la gestión real de los recursos del servidor. Ya que al principio mi idea era hacerlo con Symfony, ya que para un proyecto medianamente “grande” siempre me han dicho que es mejor apoyarte en una librería que no hacerlo a código limpio, ya que con la librería tenemos una base ya creada.

La lucha con la autenticación: Implementar el control de sesiones y el manejo del api_token fue el momento más tenso del proyecto. Gestionar las cabeceras de seguridad y asegurar que la comunicación entre Angular y PHP no dejara brechas abiertas me obligó a estudiar a fondo cómo funciona el protocolo HTTP. Fue frustrante al principio, pero ver que finalmente el sistema de login validaba usuarios de forma segura fue una de las mejores partes de este proyecto sin ninguna duda.

Lo más positivo del proyecto

Más allá de las dificultades técnicas, lo que más valor le doy al proyecto es el **panel de administración**. Me siento muy orgulloso de cómo he conseguido simplificar tareas complejas (como el CRUD de actividades) en una interfaz que se siente natural. Haber diseñado un sistema donde el administrador puede gestionar los contenidos de forma autónoma, sin tener que tocar una sola línea de código, demuestra que he logrado mi objetivo principal: crear una herramienta funcional y lista para el usuario real.

Líneas de mejora

Un proyecto de software nunca está realmente acabado, y tengo claro que, si pudiera dedicarle unos meses más, TerretaHouse daría un salto de calidad importante:

- **Implementación de pagos:** Es la pieza que falta para que la web sea 100% transaccional. Integrar una API permitiría no solo gestionar la reserva, sino también el cobro y la confirmación automática.
- **Sistema de notificaciones dinámicas:** Me gustaría que el usuario recibiera un email automático al confirmar su reserva o al cancelarla. Esto daría mucha más confianza y profesionalidad al servicio. (Pero estaríamos hablando de un desarrollo más profesional, pienso, que aún no tengo ese nivel de programador como para realizar ese sistema).

- **Buscador inteligente con filtros:** Ahora mismo el usuario navega por las actividades existentes, pero añadir un sistema de filtros (por fecha, zona geográfica o tipo de actividad) mejoraría la experiencia de usuario de forma exponencial. Esto es una espina que me gustaría haber integrado ya en este proyecto, pero que, por falta de tiempo no he podido implementar.
- **Gestión multimedia:** Actualmente, el sistema se centra en el texto y la fecha. Poder permitir que el administrador suba sus propias fotos a través de la web, y que estas se redimensionen automáticamente, sería el siguiente paso lógico para hacer la web mucho más visual y atractiva.

Reflexión final

Este proyecto me ha enseñado que el mayor valor de un programador no es saber todos los lenguajes del mundo, sino saber enfrentarse a un problema, entenderlo y buscar una solución técnica eficiente. Empecé TerretaHouse con una idea y mucho código por delante, y cierro este capítulo habiendo construido un sistema sólido, seguro y que funciona; que al fin y al cabo es lo que buscaba. Me llevo el aprendizaje de que los errores son, en realidad, los mejores profesores que he tenido durante este año.

Bibliografía

Para la elaboración y desarrollo del proyecto *TerretaHouse*, se han consultado las siguientes fuentes técnicas y documentales:

Documentación y Frameworks

Angular: <https://angular.dev/docs>

Bootstrap: <https://getbootstrap.com/docs/>

PHP: <https://www.php.net/manual/es/>

Páginas de lenguajes

W3C: <https://www.w3.org/>

Mozilla Developer Network (MDN): <https://developer.mozilla.org/es/>